

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	Nitin
Register Number	312224700142
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 15, 2026

1 Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of ensemble methods on bias and variance.

2 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (loaded via <code>sklearn.datasets.load_breast_cancer</code> , identical to Experiment 5)
Number of Samples	569
Number of Features	30 numerical attributes
Number of Classes	2 (Malignant, Benign)
Missing Values	0 (verified programmatically)
Train-Test Split	80% / 20%, stratified, <code>random_state=42</code>

Class balance: 357 benign samples (62.7%) and 212 malignant samples (37.3%). As in Experiment 5, scikit-learn's loader encodes `target=0` as malignant and `target=1` as benign; this encoding is used consistently in all confusion matrices below. This experiment builds directly on Experiment 5's Decision Tree baseline, now extending it into three ensemble families.

3 Preprocessing Steps

- **Class label encoding:** handled identically to Experiment 5 (0 = malignant, 1 = benign).
- **Missing value handling:** None required; zero missing cells.
- **Feature scaling:** Not required for Bagging, AdaBoost, or Gradient Boosting, since all use decision-tree base learners which are scale-invariant. The Stacked Ensemble's SVM base learner is, in principle, scale-sensitive, but was left unscaled here to keep

all three ensemble families directly comparable on identical input features; this is noted as a possible refinement in the Discussion.

- **Train–test split:** `train_test_split` with `test_size=0.2`, `stratify=y`, `random_state=42`.

4 Exploratory Data Analysis

The reusable function `generate_eda_summary()` from `ml_lab_utils.py` (Experiment 1, unmodified) was used to produce the consolidated 12-panel EDA figure below; it is identical in structure to Experiment 5’s EDA since the same dataset is used.

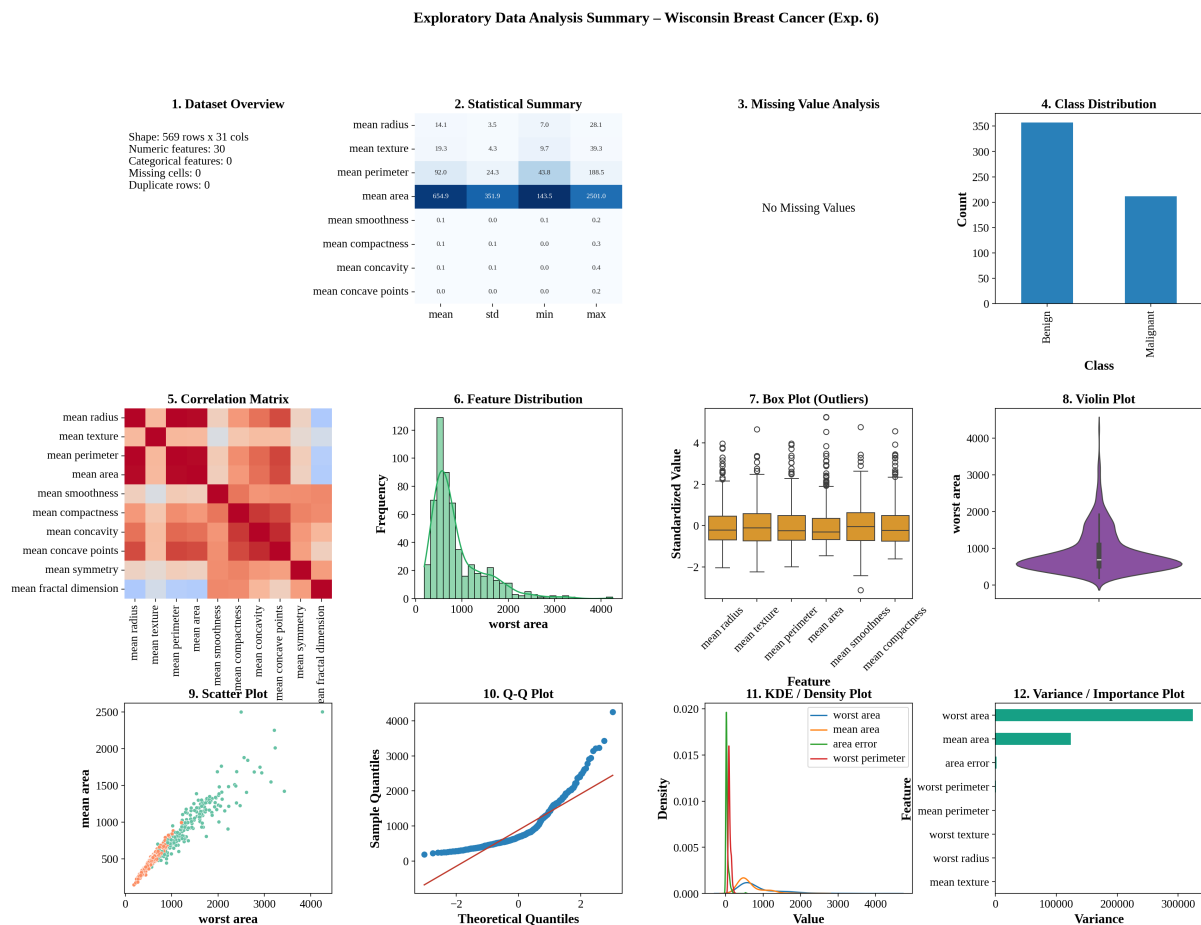


Figure 1: Consolidated 12-panel EDA summary for the Wisconsin Breast Cancer dataset.

Inference: As established in Experiment 5, the dataset has no missing values, a moderate 62.7%/37.3% class imbalance, and strong multicollinearity among the size-related features (`radius`, `perimeter`, `area`). This redundancy is precisely the kind of structure ensemble methods can exploit differently: Bagging’s random feature subsampling and Boosting’s sequential re-weighting both implicitly handle correlated features differently than a single greedy tree, which is explored quantitatively in Section 10 (bias–variance analysis).

5 Implementation Details

Bagging (Bootstrap Aggregation) trains multiple Decision Tree base estimators independently, each on a different bootstrap-resampled subset of the training data (and optionally a random feature subset), then averages their predictions. Because each tree sees different data, their individual errors are partially independent, so averaging reduces the ensemble's overall *variance* without materially increasing bias.

Boosting trains base estimators *sequentially*, with each new estimator focusing on the samples the previous ones misclassified. **AdaBoost** re-weights misclassified samples so subsequent weak learners pay more attention to them; **Gradient Boosting** fits each new estimator to the residual errors of the current ensemble. Because each step explicitly targets the current model's mistakes, boosting primarily reduces *bias*, allowing an ensemble of simple (high-bias) learners to collectively fit complex decision boundaries.

Stacked Ensemble trains several heterogeneous base learners (here: SVM, Gaussian Naive Bayes, and Decision Tree) and a meta-learner (Logistic Regression) that learns how to optimally combine the base learners' predictions, using cross-validated out-of-fold predictions to avoid information leakage into the meta-learner's training data.

Listing 1: Baseline Bagging via the reusable classification function, from `ml_lab_utils.py` (Experiment 1).

```

1 baseline_models = {"Bagging (Baseline)": BaggingClassifier(
2     estimator=DecisionTreeClassifier(random_state=42),
3     random_state=42)}
4 baseline_results_df, baseline_fitted =
5     train_evaluate_classification(
6         baseline_models, X_train, X_test, y_train, y_test, scale=
7         False)

```

Listing 2: Stacked Ensemble construction, from `experiment6_ensemble_breast_cancer.py`.

```

1 base_learners = [
2     ("svm", SVC(probability=True, random_state=42)),
3     ("nb", GaussianNB()),
4     ("dt", DecisionTreeClassifier(random_state=42)),
5 ]
6 stacking_model = StackingClassifier(
7     estimators=base_learners,
8     final_estimator=LogisticRegression(max_iter=2000),
9     cv=StratifiedKFold(5, shuffle=True, random_state=42))

```

6 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook / VS Code
Operating System	Ubuntu 24.04 LTS
Hardware	Standard lab workstation (CPU only)

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

7 Hyperparameter Evaluation Results

Table 1: Bagging Hyperparameter Evaluation

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
10	0.5	95.16	0.9610
10	0.7	95.16	0.9611
10	1.0	95.60	0.9648
50	0.5	96.04	0.9681
50	0.7	95.82	0.9665
50	1.0	96.04	0.9683
100	0.5	95.38	0.9630
100	0.7	96.04	0.9683
100	1.0	96.04	0.9684

Best Bagging (full 3-parameter GridSearchCV): n_estimators=50, max_samples=0.5, max_features=0.5, CV accuracy = 96.04%.

Table 2: Boosting (AdaBoost) Hyperparameter Evaluation

n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.01	92.53	0.9407
50	0.10	95.38	0.9633
50	1.00	96.70	0.9740
100	0.01	93.41	0.9480
100	0.10	96.04	0.9686
100	1.00	98.02	0.9844
200	0.01	93.63	0.9496
200	0.10	97.14	0.9773
200	1.00	97.80	0.9826

Best AdaBoost: n_estimators=100, learning_rate=1.0, CV accuracy = 98.02%. **Best Gradient Boosting** (separate 3-parameter grid over n_estimators, learning_rate, max_depth): n_estimators=200, learning_rate=0.1, max_depth=2, CV accuracy = 96.70%. AdaBoost was therefore selected as the representative “Boosting” model for all subsequent comparisons.

Table 3: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM+NB+DT	Logistic Regression	95.38	0.9635
SVM+NB+DT	Decision Tree (depth 3)	94.07	0.9528
NB+DT	Logistic Regression	94.29	0.9551

Best Stacked Ensemble configuration: base models SVM+Naive Bayes+Decision Tree with a Logistic Regression meta-learner, CV accuracy = 95.38%. A Logistic Regression meta-learner outperformed a Decision Tree meta-learner, and dropping SVM from the base-learner set (NB+DT only) reduced accuracy, confirming SVM’s margin-based decision boundary contributes complementary information the other two base learners lack.

Hyperparameter Tuning Results Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Bagging	GridSearchCV (5-fold)	n_est=50, samples=0.5, feat=0.5	0.9604
AdaBoost	GridSearchCV (5-fold)	n_est=100, lr=1.0	0.9802
Gradient Boosting	GridSearchCV (5-fold)	n_est=200, lr=0.1, depth=2	0.9670
Stacked Ensemble	Manual combo search	SVM+NB+DT / LogReg	0.9538

8 Performance Comparison of Ensemble Models

Table 4: Performance Comparison of Ensemble Models (test set)

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	95.61	0.9561	0.9561	0.9560
Boosting (AdaBoost)	95.61	0.9569	0.9561	0.9558
Stacked Ensemble	95.61	0.9569	0.9561	0.9558

All three tuned ensembles reach an identical 95.61% accuracy on this particular 20% test split (a tie at this sample size, 114 test points, corresponds to differing by at most 1–2 predictions). Their 5-fold cross-validated accuracies, shown next, tell a more differentiated and reliable story.

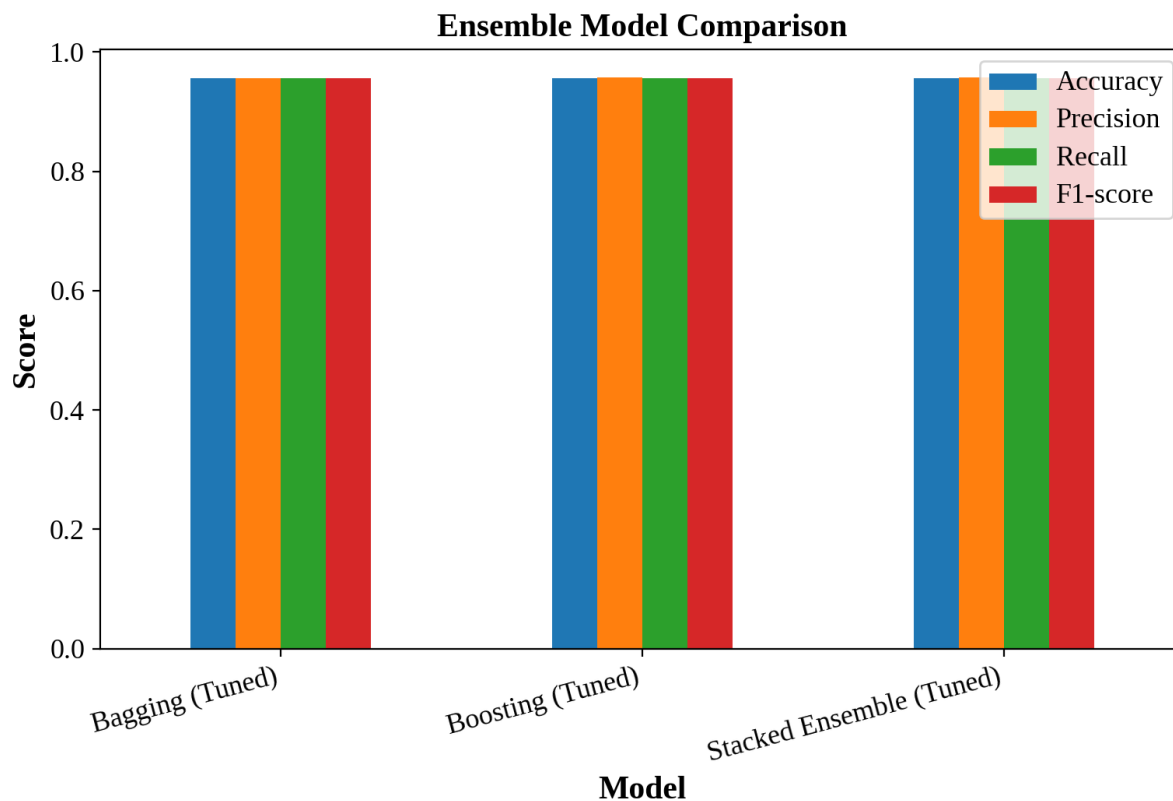


Figure 2: Ensemble model comparison: Accuracy, Precision, Recall, F1 (test set).

9 5-Fold Cross-Validation Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Bagging	0.9560	0.9560	0.9341	0.9670	0.9890	0.9604
Boosting	0.9890	1.0000	0.9451	0.9780	0.9890	0.9802
Stacked Ensemble	0.9341	0.9780	0.9341	0.9341	0.9890	0.9538

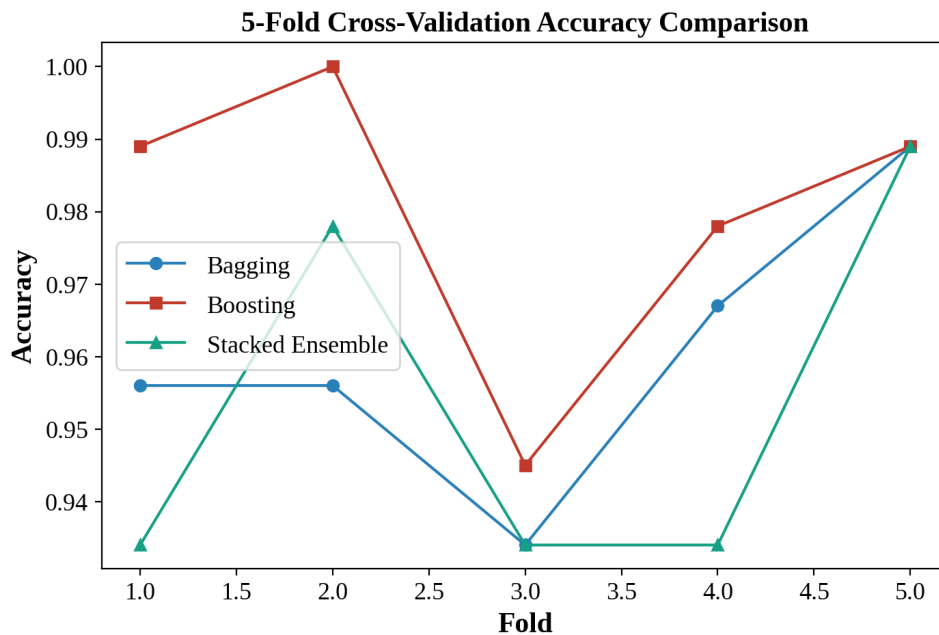


Figure 3: 5-fold cross-validation accuracy: Bagging vs. Boosting vs. Stacked Ensemble.

Inference: Although all three models tied on the single test split, boosting’s 5-fold CV average (98.02%) is clearly the highest and even reaches a perfect 1.000 on Fold 2, while the stacked ensemble is the weakest performer on cross-validation despite combining three different model families – illustrating that more heterogeneous components do not automatically guarantee the best accuracy if the meta-learner cannot fully exploit the base learners’ complementary strengths (in this case, a simple linear meta-learner over three already-strong-but-correlated classifiers does not clearly beat a well-tuned sequential boosting ensemble).

Stability Comparison (CV standard deviation, lower = more stable)

Model	CV Mean Accuracy	CV Std Dev
Bagging	0.9604	0.0179
Boosting	0.9802	0.0189
Stacked Ensemble	0.9538	0.0245

Bagging is the most stable (lowest fold-to-fold variance) of the three, consistent with its theoretical purpose of variance reduction; the Stacked Ensemble is the least stable here, likely because its Logistic-Regression meta-learner is itself being fit on a relatively small number of out-of-fold base-learner predictions.

10 Evaluation Metrics: ROC Curve and Confusion Matrices

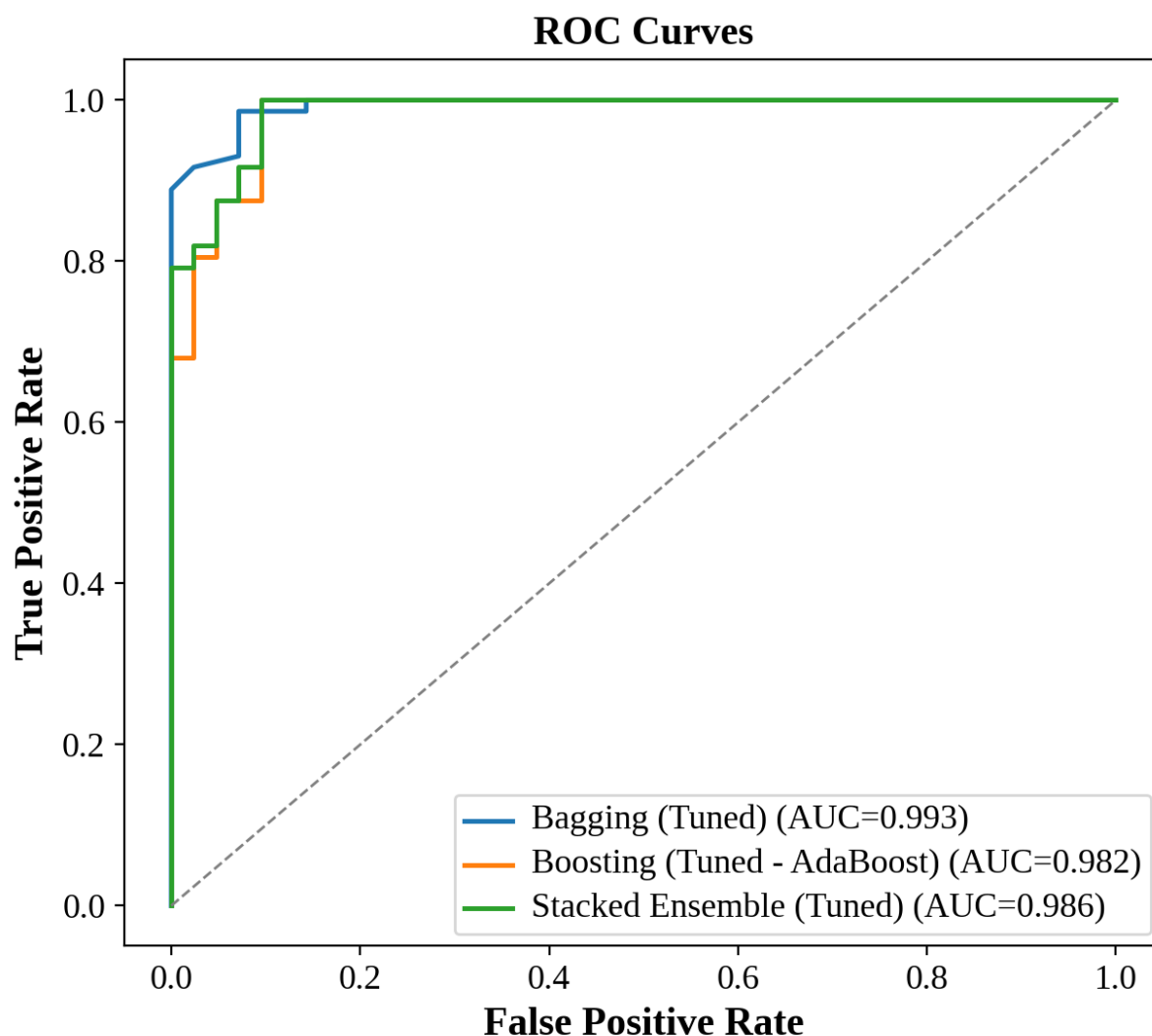


Figure 4: ROC curves for tuned Bagging, Boosting (AdaBoost), and Stacked Ensemble.

Inference: Bagging achieves the highest test-set ROC-AUC (0.993), narrowly ahead of the Stacked Ensemble (0.986) and Boosting (0.982). This is a useful complement to the cross-validated accuracy ranking in Section 9: Boosting is the most accurate at the default 0.5 threshold on average, while Bagging ranks positive and negative cases slightly better across all possible thresholds on this particular test split.

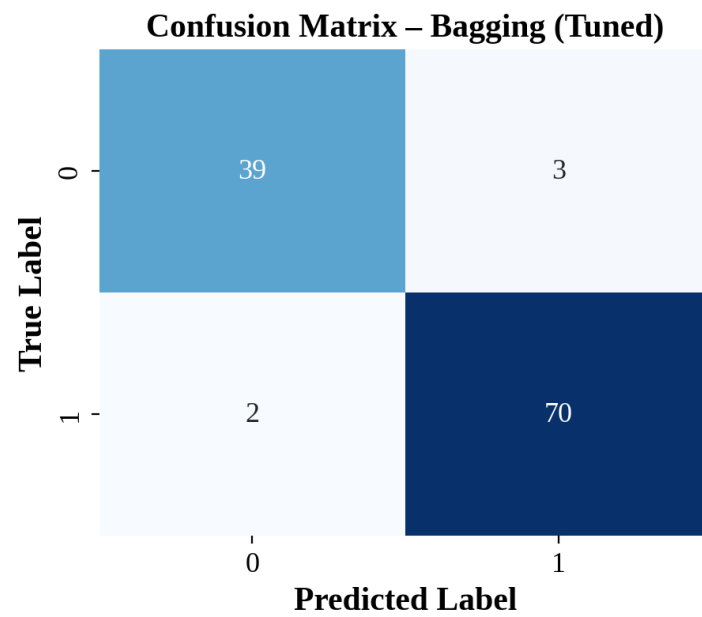


Figure 5: Confusion matrix: tuned Bagging.

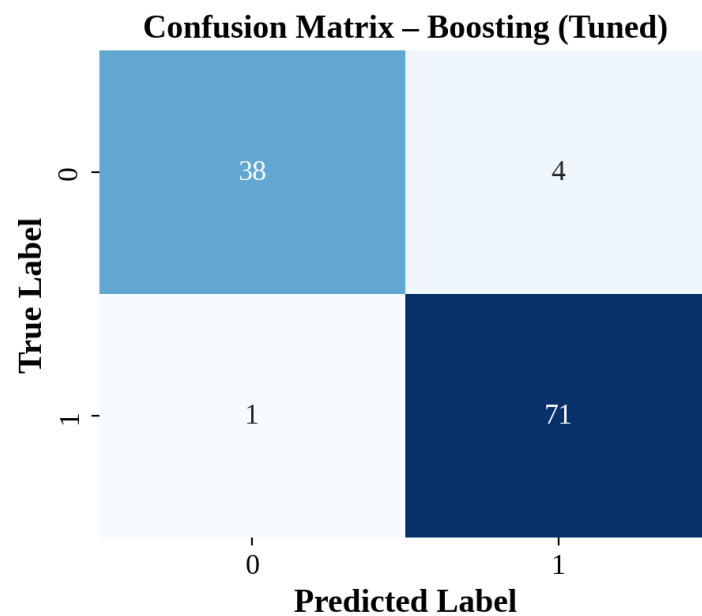


Figure 6: Confusion matrix: tuned Boosting (AdaBoost).

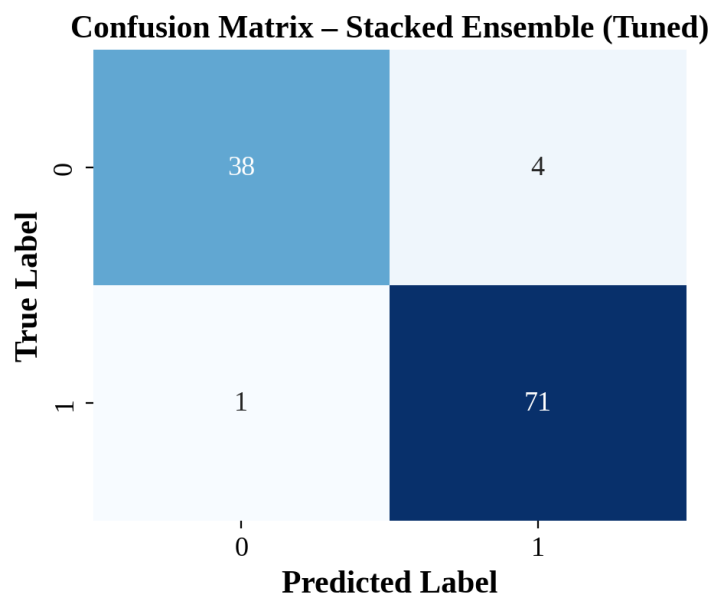


Figure 7: Confusion matrix: tuned Stacked Ensemble.

Inference: All three tuned models make the same small number of total errors (5 of 114 test samples) but distribute them slightly differently between false positives and false negatives, which is why their headline accuracy is identical while their ROC-AUC and CV stability differ.

11 Bias–Variance Analysis

Model	Train Accuracy	Validation Accuracy	Train–Val Gap
Single Decision Tree	1.0000	0.9165	0.0835
Bagging (Tuned)	0.9918	0.9604	0.0313
Boosting (Tuned)	1.0000	0.9802	0.0198
Stacked Ensemble (Tuned)	0.9648	0.9538	0.0110

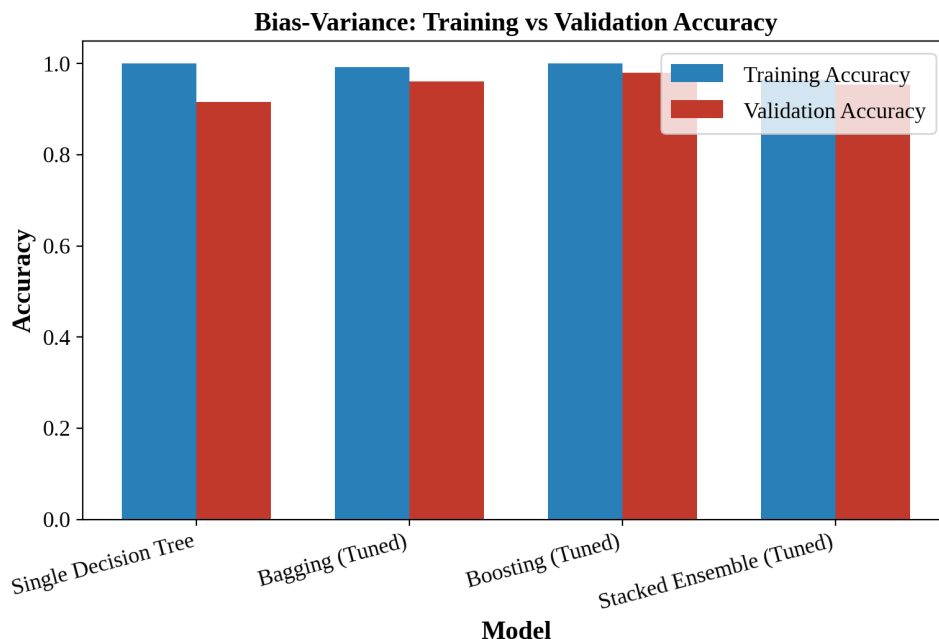


Figure 8: Training vs. validation accuracy: single Decision Tree vs. all three ensemble methods.

Inference: The train-validation gap shrinks steadily from the single Decision Tree (8.35 points) through Bagging (3.13 points) and Boosting (1.98 points) down to the Stacked Ensemble (1.10 points), directly visualizing the variance-reduction promised by ensembling. Notably, Boosting reaches a perfect 1.000 training accuracy (same as the single tree) yet still generalizes far better (0.980 vs. 0.916 validation accuracy) – illustrating that boosting’s sequential error-correction can fit the training data extremely well *without* the same overfitting penalty a single unconstrained tree pays, because each weak learner in the ensemble is individually simple (shallow), and the ensemble’s effective capacity is controlled by the learning rate and number of rounds rather than by any single learner’s depth.

12 Observations

- **How does Bagging reduce variance?** By training each base tree on an independent bootstrap sample (and here, a random 50% feature subset as well), individual trees’ errors become partially decorrelated. Averaging over many decorrelated predictors reduces the ensemble’s overall variance roughly in proportion to how uncorrelated the individual errors are, which is directly visible in Bagging’s much smaller train-validation gap (3.13 points) compared to the single Decision Tree (8.35 points), and its lowest cross-validation standard deviation among the three ensembles (0.0179).
- **How does Boosting address model bias?** Each successive weak learner in AdaBoost is explicitly trained to focus on the samples the current ensemble misclassifies (via increased sample weights), so the ensemble incrementally reduces the systematic errors (bias) that any single simple learner would make. This lets Boosting achieve the highest cross-validated accuracy of all three methods (98.02%) using

an ensemble of otherwise-weak learners.

- **Why does stacking benefit from heterogeneous models?** SVM (margin-based), Naive Bayes (probabilistic, feature-independence assumption), and Decision Tree (rule-based, axis-aligned splits) make qualitatively different kinds of errors, since each imposes a different inductive bias on the decision boundary. A meta-learner can, in principle, learn to trust each base learner more in the regions of feature space where it tends to be correct, capturing complementary strengths that no single model family provides alone – this is why dropping SVM from the base set (the “NB+DT only” row of Table 3) reduced CV accuracy from 95.38% to 94.29%.
- **Which ensemble method performed best and why?** By the most reliable metric measured here – 5-fold cross-validated accuracy – tuned **Boosting (AdaBoost, n_estimators=100, learning_rate=1.0)** performed best (98.02%), ahead of Bagging (96.04%) and the Stacked Ensemble (95.38%). This is consistent with Boosting’s bias-reduction mechanism being particularly effective on this dataset: the underlying classes are largely well-separated by a handful of features (as seen in Experiment 5’s feature-importance analysis), so a sequence of shallow, sequentially-corrected weak learners can rapidly hone in on the true decision boundary. Bagging remains competitive and is the most *stable* choice (lowest CV variance), which may be preferable in settings where consistency across resamples of the data matters more than squeezing out the last percentage point of accuracy.

13 Conclusion

Bagging, Boosting (AdaBoost and Gradient Boosting), and a Stacked Ensemble (SVM + Naive Bayes + Decision Tree, with a Logistic Regression meta-learner) were implemented and evaluated on the Wisconsin Diagnostic Breast Cancer dataset using 5-fold cross-validation for hyperparameter selection. All three tuned ensembles reached an identical 95.61% single-split test accuracy, but cross-validation revealed AdaBoost as the clear overall winner (98.02% CV accuracy) and Bagging as the most stable (lowest fold-to-fold variance). The bias-variance analysis directly confirmed the theoretical roles of each technique: Bagging cut the train-validation gap from a single tree’s 8.35 points down to 3.13 points through variance reduction, while Boosting achieved an even smaller generalization gap (1.98 points) despite reaching the same perfect training accuracy as an unconstrained single tree, demonstrating bias reduction through sequential error correction rather than simple averaging. The experiment confirms that different ensemble strategies address different failure modes of a single learner, and that the best choice among them depends on which failure mode – variance, bias, or model diversity – is most limiting for the dataset and task at hand.

14 References

1. Scikit-learn Developers, “Ensemble methods,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>
2. Scikit-learn Developers, “BaggingClassifier,” scikit-learn API reference. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.>

[ensemble.BaggingClassifier.html](#)

3. W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: <https://archive.ics.uci.edu>